

IN3030 – Oblig 1

The chip used was the M1 – chip from apple and has eight cores.

```
1  N;K;Parallel;Sequential
2  1000;100;0,980 ;0,566
3  10000;100;0,775 ;4,851
4  100000;100;3,311 ;11,521
5  1000000;100;11,265 ;109,930
6  10000000;100;4,211 ;1016,741
7  100000000;100;21,214 ;11374,393
8
9  1000;20;0,667 ;0,055
10 10000;20;0,410 ;0,566
11 100000;20;0,467 ;6,817
12 1000000;20;0,891 ;82,306
13 10000000;20;2,925 ;982,207
14 100000000;20;24,103 ;11426,944
15
16 N;K;Parallel;SequentialInsertSort
17 1000;20;0,604 ;0,028
18 10000;20;0,466 ;0,227
19 100000;20;0,480 ;1,866
20 1000000;20;0,826 ;3,875
21 10000000;20;2,639 ;13,588
22 100000000;20;20,813 ;135,530
23
```

Until a certain point, the sequence-program will be faster than the threads. This might be because of the time it takes to start threads, but when N increases the thread program starts to outperform the sequential programs with and without insertSort.

The normal sequential program uses the inbuilt sorting algorithm method in java, which is quicksort. With quicksort you will get a running time of $n * \log(n)$, but because of randomized numbers it will be close to impossible to choose the best pivot. Therefore, it will take longer than $n * \log(n)$ because of a lot of swapping and not choosing the best pivot. If we compare it to our insertSort used in our program, we will see that we don't sort through all the numbers, but rather check if they are bigger than the last element. Worst case is n^2 in running time but that is not very likely. If we find an element bigger than our arrays last element, we know it is one of the largest numbers encountered so far. The new number swaps out our last number

and we swap until it is not greater than the element in front. This way we can conserve computing time and make our program go faster. If we compare the times between Sequential with and without insertSort, we can clearly see there is a big speed difference

The same still stands even when k has increased. We also see that even tho the performance of the thread is still better than the sequential program, it is still slower than when k was 20.

I wasn't able to make a graph unfortunately...